

# REACT - ADVANCED

# AULA 01

## SUMÁRIO

|                                  |    |
|----------------------------------|----|
| O QUE VEM POR AÍ?.....           | 3  |
| HANDS ON .....                   | 4  |
| SAIBA MAIS.....                  | 4  |
| O QUE VOCÊ VIU NESTA AULA? ..... | 10 |
| REFERÊNCIAS.....                 | 11 |

EMSE

## O QUE VEM POR AÍ?

Prepare-se para uma jornada que levará suas habilidades de desenvolvimento web a um novo patamar. Vamos explorar o fascinante mundo do SSR (Server Side Rendering) e aprender a migrar um projeto React existente para o Next.

Faremos isso em 3 etapas e você será guiado(a) passo a passo por meio do processo de preparação do ambiente, implementação prática e conceitos importantes do Next.

EMSE

## HANDS ON

Vamos mergulhar em um hands-on dinâmico e prático, onde exploraremos a criação e o uso de um custom Hook, useGeolocation, para capturar a localização do usuário. As videoaulas te guiarão passo a passo, garantindo uma compreensão profunda e aplicável do conteúdo.

No primeiro vídeo, daremos o pontapé inicial na nossa jornada com uma introdução ao Next e sua importância no desenvolvimento moderno de aplicações web. Você aprenderá sobre seus benefícios, como ele melhora a performance e o SEO das suas aplicações, e porque se tornou uma técnica essencial para as pessoas desenvolvedoras. Vamos começar com uma revisão rápida do projeto React que será migrado, analisando sua estrutura de código e arquivos. Em seguida, faremos a preparação do ambiente de desenvolvimento, configurando tudo que é necessário para utilizar o SSR com o Next.js. Isso incluirá a instalação de dependências e a configuração inicial do Next. Para finalizar, escreveremos nosso primeiro componente e faremos as alterações iniciais no código, preparando o terreno para as etapas seguintes.

No segundo vídeo, migraremos os primeiros componentes. Este é o momento de colocar a mão na massa e ver o SSR em ação. Começaremos configurando o Next.js para trabalhar com styled components. Em seguida, migraremos o componente Banner do nosso projeto React com Vite para o Next.js, demonstrando como os componentes React são renderizados no servidor. Você verá em detalhes como o SSR transforma a forma como a página é carregada, melhorando significativamente a experiência do usuário.

No terceiro e último vídeo, exploraremos ainda mais conceitos do Next finalizando a migração do projeto. Você aprenderá sobre técnicas para otimizar a renderização no servidor, garantindo que sua aplicação não só funcione bem, mas também seja rápida e amigável aos motores de busca. Além disso, vamos comentar sobre alternativas ao Next.js, como Remix e Gatsby.

## SAIBA MAIS

### As regras dos Hooks

Já se perguntou como o React mantém a simplicidade e a eficiência no gerenciamento de estados e efeitos colaterais? Ou como ele permite que você crie soluções elegantes e reutilizáveis para problemas comuns? A resposta está nas regras dos hooks, um conjunto de diretrizes que garantem que os hooks sejam usados corretamente, mantendo a integridade e a performance da aplicação.

Os hooks do React, introduzidos na versão 16.8, permitem que você use estado e outras funcionalidades do React em componentes funcionais. No entanto, para garantir que os hooks funcionem corretamente, existem algumas regras fundamentais que você deve seguir:

- Chame os hooks no topo da função: nunca chame hooks dentro de loops, condições ou funções aninhadas. Sempre use hooks no nível superior da sua função React, para garantir que eles sejam chamados na mesma ordem em cada renderização.
- Chame hooks apenas em funções React: chame hooks apenas dentro de componentes funcionais do React ou em hooks personalizados. Não use hooks em funções JavaScript normais.

O hook `useState` permite que você adicione estado a componentes funcionais. No `useGeolocation`, usamos `useState` para armazenar as coordenadas geográficas, o status da permissão e possíveis erros.

O hook `useEffect` permite executar efeitos colaterais em componentes funcionais, como buscar dados ou configurar assinaturas. No `useGeolocation`, usamos `useEffect` para solicitar a permissão de geolocalização e atualizar as coordenadas em tempo real. O `useEffect` também lida com a limpeza, garantindo que a assinatura de geolocalização seja removida quando o componente for desmontado.

Seguindo as regras dos hooks, colocamos `useState` e `useEffect` no nível superior do nosso hook personalizado, garantindo que eles sejam executados corretamente a cada renderização. Isso nos permite encapsular toda a lógica de geolocalização de forma clara e reutilizável.

E falando agora especificamente de custom hooks, você pode dar uma olhada [nesse Alura+](#), onde é possível ver detalhes, dicas e conceitos sobre criar nossos próprios hooks.

## **APIs do navegador**

Agora vamos conversar sobre algo super interessante que vocês já começaram a explorar: as APIs do navegador. Sei que já aprendemos a como usar a API de geolocalização dentro de uma aplicação React, mas o mundo das APIs do navegador é muito maior e repleto de possibilidades fascinantes.

Primeiro, vamos recapitular o que são as APIs do navegador: APIs, ou Interfaces de Programação de Aplicações, são conjuntos de ferramentas e protocolos que permitem que diferentes softwares se comuniquem entre si. No contexto dos navegadores web, essas APIs nos permitem acessar funcionalidades avançadas do navegador e interagir com o hardware do dispositivo de forma simples e padronizada.

### **Geolocalização**

A API de Geolocalização nos permite obter a localização do usuário com bastante precisão, usando métodos como `getCurrentPosition` e `watchPosition`. Isso pode ser útil para uma variedade de aplicações, como mapas interativos, aplicativos de entrega, redes sociais baseadas em localização, entre outros.

Agora, vamos falar sobre outras APIs igualmente interessantes e úteis.

### **API de notificações**

A API de Notificações permite que uma aplicação web envie notificações para o usuário mesmo quando a página não está ativa. Isso pode ser ótimo para alertar o usuário sobre uma nova mensagem, uma atualização importante ou qualquer evento que necessite de atenção imediata. Utilizar essa API é simples: com permissão do usuário, podemos criar notificações personalizadas usando o objeto `Notification`.

### **API de armazenamento**

Outra API muito útil é a API de Armazenamento, que inclui o `LocalStorage` e o `SessionStorage`. Com elas, podemos armazenar dados no navegador do usuário de forma persistente (`LocalStorage`) ou temporária (`SessionStorage`). Isso é perfeito para

guardar preferências de usuário, dados de sessões e muito mais, sem a necessidade de recorrer a um servidor.

### **API de câmera e microfone**

Já pensou em criar um aplicativo que precise acessar a câmera ou o microfone do usuário? Com a API de MediaDevices, isso é possível. Esta API permite capturar mídia por meio do hardware do dispositivo. Assim, podemos criar aplicativos de videoconferência, captura de fotos e vídeos, gravação de áudio, etc. Tudo isso com apenas algumas linhas de código e, claro, com a permissão do usuário.

### **API de sensores**

Os navegadores modernos também oferecem APIs para acessar os sensores do dispositivo, como o acelerômetro, giroscópio e magnetômetro. A API de sensores pode ser usada para detectar movimentos, orientação e outras interações físicas do dispositivo, abrindo portas para aplicativos de realidade aumentada, jogos e monitoramento de atividades físicas.

### **API de pagamentos**

A API de pagamentos é uma ferramenta poderosa para quem deseja implementar processos de pagamento diretamente no navegador. Ela permite uma integração fácil e segura com diferentes métodos de pagamento, facilitando a vida dos(as) desenvolvedores(as) e proporcionando uma experiência de compra mais fluida para os usuários.

### **API de manipulação de arquivos**

Por último, mas não menos importante, temos a API de manipulação de arquivos. Ela permite que as aplicações web interajam com o sistema de arquivos do usuário, permitindo upload, download e até mesmo edição de arquivos diretamente no navegador. Isso é especialmente útil para aplicativos que lidam com documentos, imagens ou qualquer tipo de arquivo que precise ser gerenciado pelo usuário.

Como vocês podem ver, as APIs do navegador nos oferecem um vasto leque de funcionalidades, que podemos incorporar em nossas aplicações web. Elas nos permitem criar experiências mais ricas, interativas e funcionais, aproximando cada vez mais as aplicações web das nativas.

Continuem explorando e experimentando essas APIs, pois o aprendizado contínuo é a chave para se tornar uma pessoa desenvolvedora web cada vez mais competente

e inovadora. E não se esqueça: sempre respeite a privacidade e a segurança dos usuários, pedindo permissões de forma clara e utilizando as APIs de maneira ética e responsável.

### **Create React App (CRA) vs Vite**

O Create React App foi, por muito tempo, a ferramenta padrão para iniciar novos projetos React. Ele configurava automaticamente todo o ambiente de desenvolvimento, incluindo o Webpack para bundling, Babel para transpiling, e outras configurações complexas, permitindo que os profissionais de desenvolvimento se concentrassem no código React sem se preocupar com detalhes de configuração. No entanto, embora o CRA tenha sido extremamente útil, ele começou a mostrar suas limitações à medida que as necessidades e expectativas dos(as) desenvolvedores(as) evoluíram.

Uma das principais razões para a depreciação do CRA é o desempenho. Com sua configuração padrão de Webpack, ele pode ser lento em projetos maiores, especialmente durante a fase de desenvolvimento. O tempo de build e a velocidade de hot-reloading podem ser insatisfatórios. Além disso, enquanto o CRA é ótimo para iniciantes, ele pode ser restritivo para desenvolvedores(as) mais experientes que precisam de configurações personalizadas. Customizar o Webpack e outras partes do ambiente de desenvolvimento pode ser complicado. Outro ponto é a manutenção: manter e atualizar o CRA para acompanhar as novas tecnologias e melhores práticas no desenvolvimento front-end tem se tornado cada vez mais difícil.

Para solucionar muitos desses problemas, o Vite surgiu como uma alternativa moderna e eficiente. Desenvolvido por Evan You, o criador do Vue.js, o Vite foi projetado para ser um "bundler" de próxima geração, proporcionando uma experiência de desenvolvimento rápida e ágil. O Vite utiliza o ESBuild para transpilar o código, o que é significativamente mais rápido que o Babel utilizado pelo CRA. Isso resulta em tempos de inicialização quase instantâneos e hot-reloading muito mais rápido. Além disso, o Vite é configurado para funcionar imediatamente, mas também permite fácil personalização. Ele adota uma abordagem de configuração mínima, o que reduz a complexidade e torna mais fácil ajustar as configurações conforme necessário.

O Vite também suporta nativamente as últimas especificações de JavaScript e CSS, estando alinhado com as melhores práticas modernas de desenvolvimento front-end. Ele possui uma comunidade ativa e em crescimento, o que garante melhorias e



suporte contínuos. O ecossistema do Vite não se limita apenas ao React; ele suporta outros frameworks como Vue, Svelte, e até mesmo projetos vanilla JavaScript, tornando-o uma ferramenta versátil para desenvolvedores(as) que trabalham com múltiplas tecnologias.

Se você está pensando em migrar de CRA para Vite, não se preocupe, o processo é relativamente simples. Primeiro, instale o Vite em seu projeto utilizando o npm ou o yarn. O Vite cria um arquivo de configuração simples (`vite.config.js`) que você pode ajustar conforme necessário. A maioria das configurações padrão funcionará bem para a maioria dos projetos. Pode ser necessário ajustar algumas importações ou configurações específicas do CRA, mas a maioria do seu código React funcionará sem modificações. Após configurar e ajustar seu projeto, teste tudo para garantir que está funcionando conforme esperado. Em seguida, você pode continuar o desenvolvimento com todas as vantagens que o Vite oferece.

A transição do Create React App para o Vite representa um avanço significativo na experiência de desenvolvimento front-end. Com desempenho superior, maior flexibilidade e suporte a tecnologias modernas, o Vite está se estabelecendo como a nova ferramenta padrão para projetos React. Portanto, se você ainda não experimentou o Vite, esta é uma excelente oportunidade para explorar suas capacidades e melhorar seu fluxo de trabalho. Continuem aprendendo e se adaptando às novas ferramentas e tecnologias, pois isso é fundamental para se manterem atualizados(as) e eficientes no mundo em constante evolução do desenvolvimento web.

## O QUE VOCÊ VIU NESTA AULA?

Durante essa jornada, você explorou a implementação prática de geolocalização em React, adquirindo habilidades essenciais para criar funcionalidades avançadas em suas aplicações. Iniciamos configurando o ambiente de desenvolvimento e criando um projeto React utilizando Node.js e Vite, estabelecendo a base para nosso trabalho. Em seguida, aprendemos a criar um hook personalizado, `useGeolocation`, para gerenciar a localização do usuário.

Utilizamos `useState` para armazenar as coordenadas e o estado da permissão, e `useEffect` para acessar a API de Geolocalização, garantindo que nossa aplicação pudesse reagir a mudanças na localização em tempo real. Adicionamos lógica para gerenciar os estados de permissão com `navigator.permissions`, permitindo que nossa aplicação respondesse adequadamente às permissões concedidas ou negadas pelos usuários.

Também implementamos funções para manipular os resultados da API de Geolocalização, tratando tanto os casos de sucesso quanto os erros, assegurando que nossa aplicação fosse robusta e capaz de lidar com diferentes cenários. Por meio de testes no navegador, verificamos que nosso hook funcionava corretamente e que os estados de erro estavam sendo tratados de maneira eficaz.

Para consolidar o aprendizado, implementamos um componente React que utilizava o hook `useGeolocation` para exibir a localização e o status de permissão, demonstrando como a aplicação deveria se comportar em diversos cenários de permissão. Além disso, discutimos a importância da programação defensiva ao lidar com APIs que requerem permissões, apresentando estratégias para lidar com erros e manter o código seguro e robusto.

Revisando esses conceitos e práticas, agora você tem uma compreensão sólida de como integrar a geolocalização em suas aplicações React, utilizando hooks personalizados de maneira eficaz. Esta experiência não só ampliou suas habilidades técnicas, mas também reforçou a importância de seguir boas práticas de programação para garantir a qualidade e a confiabilidade de seu código.

## REFERÊNCIAS

MDN WEB DOCS. **Geolocation API**. 2024. Disponível em: [https://developer.mozilla.org/en-US/docs/Web/API/Geolocation\\_API](https://developer.mozilla.org/en-US/docs/Web/API/Geolocation_API). Acesso em: 01 jul. 2024.

REACT. **Using the Effect Hook**. 2023. Disponível em: <https://react.dev/reference/react/useEffect>. Acesso em: 01 jul. 2024.

REACT. **Reusing Logic with Custom Hooks**. 2023. Disponível em: <https://react.dev/learn/reusing-logic-with-custom-hooks>. Acesso em: 01 jul. 2024.

EMANUS

## **PALAVRAS-CHAVE**

Geolocalização, React Hooks, Custom Hooks, useState, useEffect, Permissões, Programação Defensiva, API de Geolocation.

EMSE

# POSTECH